Cache Control

1.0

Создано системой Doxygen 1.9.8

1 CacheControl — система управления страничным кэшем 1
1.1 Содержание . . . 1
1.2 Возможности . . . 1
1.2.1 Требования . . . 2
1.3 Конфигурация . . . 2
1.3.1 Формат конфигурационного файла . . . 2
1.3.2 Поддерживаемые политики . . . 2
1.3.3 Параметры . . . 2
1.3.4 Пример конфигурационного файла . . . 2
1.4 Уровни логирования . . . 3
1.5 Использование . . . 3
1.5.1 Запуск демона . . . 3
1.5.2 Запуск медиатора . . . 3
1.6 Команды медиатора . . . 3
1.7 Сигналы . . . 3
1.8 Структура проекта . . . 4
1.8.1 Модули демона (src/cache_control/) . . . 4
1.8.2 Модули медиатора (src/cache_control_mediator/) . . . 5
1.9 Сборка . . . 5
1.9.1 Доступные цели Make . . . 5
1.9.2 Сборка проекта . . . 5
1.9.3 Генерация документации . . . 5
1.10 Документация . . . 6

2 Topic Index 7
2.1 Topics . . . 7

3 Алфавитный указатель структур данных 9
3.1 Структуры данных . . . 9

4 Список файлов 11
4.1 Файлы . . . 11

5 Topic Documentation 13
5.1 Contex . . . 13
5.1.1 Подробное описание . . . 13
5.1.2 Функции . . . 13
5.1.2.1 init_context() . . . 13
5.1.2.2 remove_context() . . . 14
5.2 Daemonize . . . 14
5.2.1 Подробное описание . . . 14
5.2.2 Функции . . . 14
5.2.2.1 daemonize() . . . 14
5.3 Event_stuff . . . 15
5.3.1 Подробное описание . . . 15

5.3.2 Функции . . . 15
5.3.2.1 add_fd_to_epoll() . . . 15
5.3.2.2 set_timer() . . . 15
5.4 File_loading . . . 16
5.4.1 Подробное описание . . . 16
5.4.2 Функции . . . 16
5.4.2.1 load_file() . . . 16
5.4.2.2 map_file() . . . 17
5.4.2.3 unmap_file() . . . 17
5.5 File_monitoring . . . 17
5.5.1 Подробное описание . . . 18
5.5.2 Функции . . . 18
5.5.2.1 check_type_fd() . . . 18
5.5.2.2 do_file_loading() . . . 19
5.5.2.3 do_file_statistics() . . . 19
5.5.2.4 init_file_monitoring() . . . 20
5.5.2.5 map_all_files() . . . 20
5.5.2.6 processing_all_files() . . . 20
5.5.2.7 reload_config() . . . 21
5.5.2.8 start_file_monitoring() . . . 21
5.6 File_statistics . . . 22
5.6.1 Подробное описание . . . 22
5.6.2 Функции . . . 22
5.6.2.1 get_file_size() . . . 22
5.6.2.2 get_last_access() . . . 22
5.6.2.3 get_number_of_pages() . . . 23
5.6.2.4 get_number_of_pages_cache() . . . 23
5.6.2.5 get_page_stat() . . . 24
5.6.2.6 get_percent_cache() . . . 24
5.7 Inotify_stuff . . . 24
5.7.1 Подробное описание . . . 25
5.7.2 Функции . . . 25
5.7.2.1 add_all_files_to_inotify() . . . 25
5.7.2.2 file_in_attrib() . . . 25
5.7.2.3 file_in_modify() . . . 26
5.7.2.4 read_inotify() . . . 26
5.7.2.5 remove_all_files_from_inotify() . . . 27
5.8 Options . . . 27
5.8.1 Подробное описание . . . 27
5.8.2 Функции . . . 27
5.8.2.1 parse_options() . . . 27
5.8.2.2 print_info() . . . 28
5.8.2.3 validate_path_config() . . . 28

5.9 Parse_config . . . 28
5.9.1 Подробное описание . . . 29
5.9.2 Макросы . . . 29
5.9.2.1 DEFAULT_INTERVAL . . . 29
5.9.2.2 SIZE_BUFF . . . 29
5.9.3 Функции . . . 29
5.9.3.1 parse_config() . . . 29
5.10 Signal_handler . . . 30
5.10.1 Подробное описание . . . 30
5.10.2 Функции . . . 30
5.10.2.1 init_signal_handler() . . . 30
5.10.2.2 register_handler() . . . 31
5.10.2.3 set_context() . . . 31
5.10.2.4 sighup_handler() . . . 31
5.10.2.5 sigint_handler() . . . 32
5.10.2.6 sigterm_handler() . . . 32
5.10.2.7 sigusr1_handler() . . . 32
5.11 Threads . . . 32
5.11.1 Подробное описание . . . 33
5.11.2 Функции . . . 33
5.11.2.1 start_threads() . . . 33
5.11.2.2 stop_threads() . . . 33
5.12 control mediator . . . 33
5.12.1 Подробное описание . . . 35
5.12.2 Макросы . . . 35
5.12.2.1 CLEAR_TERMINAL . . . 35
5.12.2.2 COMMAND_LEN . . . 35
5.12.2.3 MAX_TOK . . . 35
5.12.2.4 N . . . 35
5.12.2.5 PATH_BUFF_SIZE . . . 35
5.12.2.6 PATH_LEN . . . 36
5.12.2.7 PATH_LOG_FILE . . . 36
5.12.2.8 POLITIC_LEN . . . 36
5.12.2.9 RECV_FILE . . . 36
5.12.2.10 RED . . . 36
5.12.2.11 SEND_FILE . . . 36
5.12.3 Перечисления . . . 36
5.12.3.1 command_type_t . . . 36
5.12.3.2 pol . . . 37
5.12.4 Функции . . . 37
5.12.4.1 clear_stdin() . . . 37
5.12.4.2 get_command() . . . 37
5.12.4.3 get_command_type() . . . 38

5.12.4.4 get_politic_str() . . . 38
5.12.4.5 open_queue() . . . 38
5.12.4.6 print_answer() . . . 39
5.12.4.7 print_menu() . . . 39
5.12.4.8 read_string() . . . 39
5.12.4.9 set_command() . . . 40
5.12.4.10 split_tokens() . . . 40
5.13 List . . . 41
5.13.1 Подробное описание . . . 42
5.13.2 Перечисления . . . 42
5.13.2.1 politics . . . 42
5.13.3 Функции . . . 42
5.13.3.1 add_file() . . . 42
5.13.3.2 add_file_stat() . . . 42
5.13.3.3 delete_file_by_path() . . . 43
5.13.3.4 delete_file_by_wd() . . . 43
5.13.3.5 find_file_by_path() . . . 44
5.13.3.6 find_file_by_wd() . . . 44
5.13.3.7 free_list() . . . 44
5.13.3.8 init_empty_stat() . . . 45
5.13.3.9 init_file() . . . 45
5.13.3.10 init_file_stat() . . . 45
5.13.3.11 init_list() . . . 46
5.13.3.12 init_node() . . . 46
5.14 Queue . . . 47
5.14.1 Подробное описание . . . 47
5.14.2 Макросы . . . 47
5.14.2.1 CHANGE_SIZE_MSG . . . 47
5.14.2.2 MAX_SIZE_MSG . . . 47
5.14.2.3 MQ_TIMEOUT . . . 48
5.14.2.4 SIZE_RCV . . . 48
5.14.2.5 SIZE_SND . . . 48
5.14.3 Функции . . . 48
5.14.3.1 create_queue() . . . 48
5.14.3.2 recive_msg() . . . 48
5.14.3.3 recive_msg_for_mediator() . . . 49
5.14.3.4 send_msg() . . . 49

6 Структуры данных 51
6.1 Структура cache_control . . . 51
6.1.1 Подробное описание . . . 51
6.2 Структура command_t . . . 51
6.2.1 Подробное описание . . . 52

6.2.2 Поля . . . 52
6.2.2.1 command . . . 52
6.2.2.2 path_to_file . . . 52
6.3 Структура file . . . 52
6.3.1 Подробное описание . . . 52
6.4 Структура file_info . . . 53
6.4.1 Подробное описание . . . 53
6.4.2 Поля . . . 53
6.4.2.1 age . . . 53
6.4.2.2 last_access . . . 53
6.4.2.3 number_of_pages . . . 53
6.4.2.4 number_of_pages_cache . . . 54
6.4.2.5 path . . . 54
6.4.2.6 percent_cache . . . 54
6.4.2.7 politic . . . 54
6.4.2.8 size . . . 54
6.5 Структура file_monitoring . . . 54
6.5.1 Подробное описание . . . 55
6.6 Структура file_stat . . . 55
6.6.1 Подробное описание . . . 55
6.7 Структура list . . . 55
6.7.1 Подробное описание . . . 56
6.8 Структура mediator_command . . . 56
6.8.1 Поля . . . 56
6.8.1.1 com . . . 56
6.8.1.2 path_to_file . . . 56
6.9 Структура mqfd_t . . . 56
6.9.1 Подробное описание . . . 57
6.9.2 Поля . . . 57
6.9.2.1 fds . . . 57
6.10 Структура node . . . 57
6.10.1 Подробное описание . . . 57
6.11 Структура replay_t . . . 57
6.11.1 Подробное описание . . . 58
6.11.2 Поля . . . 58
6.11.2.1 f . . . 58
6.11.2.2 last_msg . . . 58
6.11.2.3 status . . . 58
6.12 Структура reply_t . . . 58
6.12.1 Подробное описание . . . 58

7 Файлы 59
7.1 command_listener.h . . . 59

7.2 context.h . . . 59
7.3 daemonize.h . . . 60
7.4 events_stuff.h . . . 60
7.5 file_loading.h . . . 60
7.6 file_monitoring.h . . . 61
7.7 file_statistics.h . . . 61
7.8 inotify_stuff.h . . . 62
7.9 options.h . . . 62
7.10 Файл include/cache_control/parse_config.h . . . 62
7.10.1 Подробное описание . . . 63
7.11 parse_config.h . . . 63
7.12 signal_handler.h . . . 63
7.13 threads.h . . . 64
7.14 Файл include/cache_control_mediator/cache_control_mediator.h . . . 64
7.15 cache_control_mediator.h . . . 64
7.16 list.h . . . 65
7.17 log.h . . . 66
7.18 queue.h . . . 67

Предметный указатель 69

Глава 1

CacheControl — система управления страничным кэшем

CacheControl — это фоновый демон, который следит за заданными файлами или каталогами и управляет их присутствием в оперативной памяти (page cache). Демон может принудительно подгружать файлы в кэш для ускорения доступа (например, перед запуском тяжёлого приложения) или выгружать малоиспользуемые файлы, освобождая память.

1.1 Содержание

- Возможности
- Конфигурация
- Уровни логирования
- Использование
- Команды медиатора
- Сигналы
- Структура проекта
- Сборка
- Документация

1.2 Возможности

- Три политики кэширования:
  - keep — постоянно держать файл(ы) в кэше
  - evict — выгружать, если файл не использовался заданное время
  - monitor — только собирать статистику, не вмешиваясь
- Работа с масками — поддержка шаблонов в путях к файлам (/var/log/∗.log)
- Мониторинг страничного кэша — использование системного вызова mincore() для анализа присутствия страниц в памяти

- Отслеживание изменений — мониторинг файлов через inotify (события: изменение, удаление, перемещение)
- Управление через очередь сообщений — POSIX очередь для отправки команд демону
- Обработка сигналов — корректное завершение, перезагрузка конфигурации, отладочный вывод статистики
- Логирование — все события и ошибки направляются в syslog

1.2.1 Требования

- Компилятор GCC/Clang с поддержкой C11
- POSIX-совместимая ОС (Linux)

1.3 Конфигурация

1.3.1 Формат конфигурационного файла

<путь_с_маской> <политика> [параметры]

1.3.2 Поддерживаемые политики

Политика	Описание
keep	Постоянно держать файл(ы) в кэше (периодически подгружать при необходимости)
evict	Выгружать из кэша, если файл не использовался в течение заданного времени
monitor	Только собирать статистику, не вмешиваясь

1.3.3 Параметры

- age=<секунды> — время в секундах, после которого файл будет выгружен (для политики evict)
- interval=<секунды> — интервал сканирования (по умолчанию 60 секунд)

1.3.4 Пример конфигурационного файла

Файл /etc/cachecontrol.conf:

```
# База данных MySQL — всегда в кэше
/var/lib/mysql/* keep

# Логи выгружаются через час
/var/log/*.log evict age=3600

# Временные файлы выгружаются через 5 минут
/tmp/* evict age=300

# Файлы данных — только мониторинг
/opt/data/*.dat monitor

# Интервал сканирования
interval=120
```

1.4 Уровни логирования

Все сообщения демона записываются в syslog.

Уровень	Назначение
LOG_INFO	Информационные сообщения
LOG_WARNING	Предупреждения
LOG_ERR	Критические ошибки

1.5 Использование

1.5.1 Запуск демона

```
# Запуск с конфигом по умолчанию (/etc/cachecontrol.conf)
./bin/cache_control -d

# Запуск с указанием конфигурационного файла
./bin/cache_control -c /etc/cachecontrol.conf
```

1.5.2 Запуск медиатора

```
# Интерактивный режим управления демоном
./bin/cache_control_mediator
```

1.6 Команды медиатора

Команда	Аргумент	Описание
status	all	Вывести статистику по файлу или по всем отслеживаемым
cache	<file>	Принудительно подгрузить файл в кэш (игнорируя политику)
evict	<file>	Принудительно выгрузить файл из кэша
reload	—	Перечитать конфигурационный файл без остановки
exit	—	Корректно завершить работу демона
clear	—	Очистить терминал

1.7 Сигналы

Демон поддерживает обработку следующих сигналов:

Сигнал	Действие
SIGTERM / SIGINT	Корректное завершение работы (освобождение памяти, закрытие дескрипторов, удаление PID-файла)
SIGHUP	Перезагрузка конфигурационного файла без остановки демона
SIGUSR1	Вывод текущей статистики в syslog (для отладки)

Пример отправки сигналов:

```
# Перезагрузить конфигурацию
```

```
kill -SIGHUP <PID>

# Вывести статистику в syslog
kill -SIGUSR1 <PID>

# Завершить демон
kill <PID>
kill -SIGINT <PID>
```

1.8 Структура проекта

```
.
|-- bin/                        # Исполняемые файлы
|   |-- cache_control           # Демон
|   `-- cache_control_mediator  # Медиатор
|-- config/                     # Конфигурационные файлы
|   `-- cachecontrol.conf       # Пример конфига
|-- docs/                       # Документация (Doxygen)
|   |-- html/                   # Сгенерированная HTML-документация
|   `-- mainpage.md             # Главная страница документации
|-- include/                    # Заголовочные файлы (.h)
|   |-- cache_control/
|   |   |-- cache_control.h
|   |   |-- command_listener.h
|   |   |-- context.h
|   |   |-- daemonize.h
|   |   |-- events_stuff.h
|   |   |-- file_loading.h
|   |   |-- file_monitoring.h
|   |   |-- file_statistics.h
|   |   |-- inotify_stuff.h
|   |   |-- options.h
|   |   |-- parse_config.h
|   |   |-- signal_handler.h
|   |   `-- threads.h
|   |-- cache_control_mediator/
|   |   `-- cache_control_mediator.h
|   |-- list/
|   |   `-- list.h
|   |-- log/
|   |   `-- log.h
|   `-- queue/
|       `-- queue.h
|-- obj/                        # Объектные файлы (.o)
|   |-- cache_control/
|   |-- cache_control_mediator/
|   |-- list/
|   |-- log/
|   `-- queue/
|-- src/                        # Файлы с исходным кодом (.c)
|   |-- cache_control/
|   |   |-- command_listener.c
|   |   |-- context.c
|   |   |-- daemonize.c
|   |   |-- events_stuff.c
|   |   |-- file_loading.c
|   |   |-- file_monitoring.c
|   |   |-- file_statistics.c
|   |   |-- inotify_stuff.c
|   |   |-- main.c
|   |   |-- options.c
|   |   |-- parse_config.c
|   |   |-- signal_handler.c
|   |   `-- threads.c
|   |-- cache_control_mediator/
|   |   `-- main.c
|   |-- list/
|   |   `-- list.c
|   |-- log/
|   |   `-- log.c
|   `-- queue/
|       `-- queue.c
|-- Doxyfile
|-- Makefile
```

1.8.1 Модули демона (src/cache_control/)

Файл	Описание
main.c	Точка входа демона
parse_config.c	Парсер конфигурационного файла
context.c	Управление глобальным состоянием демона
daemonize.c	Запуск в фоновом режиме
signal_handler.c	Обработка сигналов
file_monitoring.c	Управление политиками кэширования
file_statistics.c	Сбор статистики через mincore()
file_loading.c	Управление страничным кэшем файлов
inotify_stuff.c	Мониторинг изменений файловой системы
command_↩ listener.c	Обработка команд медиатора
threads.c	Управление потоками демона
options.c	Обработка аргументов командной строки
events_stuff.c	Обработка событий файловой системы

1.8.2 Модули медиатора (src/cache_control_mediator/)

Файл	Описание
main.↩ c	Интерфейс управления демоном и отправка команд через очередь сообщений

1.9 Сборка

1.9.1 Доступные цели Make

```
make help          # Показать все доступные цели
make all           # Собрать все бинарные файлы
make clean         # Удалить объектные файлы
make clang_format  # Отформатировать исходный код с помощью clang-format
make docs          # Сгенерировать документацию
make docs_clean    # Удалить сгенерированную документацию
```

1.9.2 Сборка проекта

```
# Собрать демон и медиатор
make

# Собрать только демон
make cache_control

# Собрать только медиатор
make cache_control_mediator

# Удалить артефакты сборки
make clean
```

1.9.3 Генерация документации

```
# Сгенерировать HTML- и PDF-документацию
make docs

# Удалить сгенерированную документацию
make docs_clean
```

1.10 Документация

После выполнения
make docs

документация будет доступна:

- HTML: docs/html/index.html
- PDF: docs/latex/refman.pdf (если установлен LaTeX)

Проект документирован с использованием Doxygen. Документация включает:

- описание всех модулей проекта;
- описание структур данных;
- описание функций и их параметров;
- зависимости между модулями;

Глава 2

Topic Index

2.1 Topics

Here is a list of all topics with brief descriptions:

Contex 13
Daemonize 14
Event_stuff 15
File_loading 16
File_monitoring 17
File_statistics 22
Inotify_stuff 24
Options 27
Parse_config 28
Signal_handler 30
Threads 32
control mediator 33
List 41
Queue 47

Глава 3

Алфавитный указатель структур данных

3.1 Структуры данных

Структуры данных с их кратким описанием.

cache_control 51
command_t
Структура, представляющая команду управления кэшем 51
file 52
file_info
Структура, которая содержит статистику о файле 53
file_monitoring 54
file_stat 55
list 55
mediator_command 56
mqfd_t
Структура для хранения дескрипторов очередей сообщений 56
node 57
replay_t 57
reply_t
Структура, которая содержит ответ от демона на запрос медиатора 58

Глава 4

Список файлов

4.1 Файлы

Полный список документированных файлов.

include/cache_control/command_listener.h . . . 59
include/cache_control/context.h . . . 59
include/cache_control/daemonize.h . . . 60
include/cache_control/events_stuff.h . . . 60
include/cache_control/file_loading.h . . . 60
include/cache_control/file_monitoring.h . . . 61
include/cache_control/file_statistics.h . . . 61
include/cache_control/inotify_stuff.h . . . 62
include/cache_control/options.h . . . 62
include/cache_control/parse_config.h
Заголовочный файл парсера конфигурации кэширования . . . 62
include/cache_control/signal_handler.h . . . 63
include/cache_control/threads.h . . . 64
include/cache_control_mediator/cache_control_mediator.h
Заголовочный файл модуля управления кэшем через очередь сообщений . . . 64
include/list/list.h . . . 65
include/log/log.h . . . 66
include/queue/queue.h . . . 67

Глава 5

Topic Documentation

5.1 Contex

Структуры данных

- struct cache_control

Макросы

- #define PATH_SIZE 255 /* Максимальный размер пути к конфигурационному файлу */

Функции

- int init_context (struct cache_control *cc)
  Инициализирует структуру контекста программы (cache_control)
- int remove_context (struct cache_control *cc)
  Освобождает ресурсы, связанные со структурой cache_control.

5.1.1 Подробное описание

Функции, которые отвечаю за инициализацию и удаление контекста программы

5.1.2 Функции

5.1.2.1 init_context()

```
int init_context (
                struct cache_control * cc )
```

Инициализирует структуру контекста программы (cache_control)

Аргументы

struct	cache_control *cc - указатель на структуру

Возвращает

Возвращает 0 в случае успеха или -1 в случае ошибки

Устанавливает значения полей по умолчанию, инициализирует мьютекс

5.1.2.2 remove_context()

```
int remove_context (
            struct cache_control * cc )
```

Освобождает ресурсы, связанные со структурой cache_control.

Аргументы

struct	cache_control *cc - указатель на структуру

Возвращает

Возвращает 0 в случае успеха или -1 в случае ошибки

Освобождает список файлов, если он был выделен Граф вызовов:

5.2 Daemonize

Функции

- int daemonize ()
  Запускает программу в режиме демона

5.2.1 Подробное описание

Функция для демонизации процесса

5.2.2 Функции

5.2.2.1 daemonize()

```
int daemonize ( )
```

Запускает программу в режиме демона

Возвращает

Возвращает 0 в случае успеха, -1 в случае ошибки

Функция использует системный вызов daemon(0,0), который отсоединяет процесс от терминала, меняет текущую рабочую директорию на корневую и запускает работу в фоновом режиме

5.3 Event_stuff

Макросы

- #define TIMEOUT 1000 /* время, через которое epoll перестанет ждать сигнал */
- #define MAX_NUM_OF_EVENTS 2 /* количество событий, которое может произойти одновремено */

Функции

- int set_timer (int timer_fd, time_t seconds)
  Задёт время для таймера и запускает его.
- int add_fd_to_epoll (int epoll_fd, int fd)
  Добавляет файловый дескриптор в epoll список.

5.3.1 Подробное описание

event_stuff - функции, связанные с событиями

5.3.2 Функции

5.3.2.1 add_fd_to_epoll()

```
int add_fd_to_epoll (
                int epoll_fd,
                int fd )
```

Добавляет файловый дескриптор в epoll список.

Аргументы

int	epoll_fd - фаловый дескриптор epoll
int	fd - файловый дескриптор, который будет добавлен

Возвращает

Возвращает в случае успеха 0, в случае провала -1.

Граф вызова функции:

5.3.2.2 set_timer()

```
int set_timer (
                int timer_fd,
                time_t seconds )
```

Задёт время для таймера и запускает его.

Аргументы

int	timer_fd - файловый дескриптор таймера
time_t	seconds - колличество секунд

Возвращает

Возвращает в случае успеха вернётся 0, в случае провала -1.

Граф вызова функции:

5.4 File_loading

Функции, связанные с загрузкой и выгрузкой файла в оперативной памяти

Функции

- int map_file (struct file *myfile)
  Выделяет виртуальный адрес для файла
- int load_file (struct file *myfile)
  Загружает файл из виртуальной памяти в оперативную
- int unmap_file (struct file *myfile)
  Выгружает файл из виртуальной памяти, тем самым высвобождая его из опреативной памяти

5.4.1 Подробное описание

Функции, связанные с загрузкой и выгрузкой файла в оперативной памяти

sdfsdfsfd

5.4.2 Функции

5.4.2.1 load_file()

int load_file (
struct file * myfile)

Загружает файл из виртуальной памяти в оперативную

Аргументы

struct	file *myfile - файл, который будет загружен в память

Возвращает

Возвращает в случае успеха вернётся 0, в случае провала -1.

Граф вызова функции:

5.4.2.2 map_file()

int map_file (
struct file * myfile)

Выделяет виртуальный адрес для файла

Аргументы

struct	file *myfile - файл, для которого будет выделяться память

Возвращает

Возвращает в случае успеха вернётся 0, в случае провала -1.

Граф вызова функции:

5.4.2.3 unmap_file()

int unmap_file (
struct file * myfile)

Выгружает файл из виртуальной памяти, тем самым высвобождая его из опреативной памяти

Аргументы

struct	file *myfile - файл, который будет загружен в память

Возвращает

Возвращает в случае успеха вернётся 0, в случае провала -1.

Граф вызова функции:

5.5 File_monitoring

Структуры данных

- struct file_monitoring

Макросы

- #define MIN_PERCENT_CACHE 80 /* минимальный процент кэша для политики EVICT */

Функции

- int init_file_monitoring (struct file_monitoring *fm)
  Инициализирует структуру мониторинга файлов
- void * start_file_monitoring (void *args)
  Запускает процесс мониторинга файлов, используется в соотвествующем потоке
- int do_file_statistics (struct file *myfile)
  Собирает статистику о файле
- int do_file_loading (struct file *myfile)
  Применяет соотвествующую политику к файлу
- int map_all_files (struct list *list_files)
  Выделяет всем файлам в списке свой виртуальный адрес.
- int processing_all_files (struct list *list_files)
  Обрабатывает все файлы: собирает статистику и применяет политику.
- int check_type_fd (int fd, struct file_monitoring *fm, struct list *file_list)
  Проверяет тип принятого файлового дескриптора
- int reload_config (struct cache_control *cc, struct file_monitoring *fm)
  Перезагружает систему

5.5.1 Подробное описание

Функции, связанные с мониторингом файлов

5.5.2 Функции

5.5.2.1 check_type_fd()

```
int check_type_fd (
            int fd,
            struct file_monitoring * fm,
            struct list * file_list )
```

Проверяет тип принятого файлового дескриптора

Аргументы

int	fd - принятый файловый дескриптор
struct	file_monitoring *fm - структура мониторинга файлов
struct	list *file_list - список файлов

Возвращает

Возвращает в случае успеха вернётся 0, в случае провала -1.

В случае если был принят файлер вызовет processing_all_files(). В случае если это оказался inotify вызовет read_inotify(). Граф вызовов: Граф вызова функции:

5.5.2.2 do_file_loading()

int do_file_loading (
struct file * myfile)

Применяет соотвествующую политику к файлу

Аргументы

struct	file *myfile - файл, по отношении к которому будет применяться политика

Возвращает

Возвращает в случае успеха вернётся 0, в случае провала -1.

Применяемые политики:

- KEEP - проверяет процент занятых страниц. Если их меньше 80, то подгружает.
- EVICT - проверяет сколько секунд назад использовался файл. Если секунд больше значения age, выгружает файл. В случае, если выгруженный файл использовался вновь, обратно вгружает.
- MONITOR - просто выводит лог.

Граф вызовов: Граф вызова функции:

5.5.2.3 do_file_statistics()

int do_file_statistics (
struct file * myfile)

Собирает статистику о файле

Аргументы

struct	file *myfile - файл, о котором будет собираться статистика

Возвращает

Возвращает в случае успеха вернётся 0, в случае провала -1.

Сначала собирает информацию о последнем доступе, так как она может понадобиться для подгрузка evict файла. Затем проверяет наличие виртуального адреса. Если указатель на виртуальный адрес равен NULL, функция возвращается. В ином случае дополнительно собирает информация о страницах. Граф вызовов: Граф вызова функции:

5.5.2.4 init_file_monitoring()

```
int init_file_monitoring (
        struct file_monitoring * fm )
```

Инициализирует структуру мониторинга файлов

Аргументы

struct	file_monitoring *fm - указатель на структуру, которую нужно инициализировать

Возвращает

Возвращает в случае успеха вернётся 0, в случае провала -1.

Во время инициализации выделяет файловый дескриптор для epoll, таймера и inotify. Граф вызова функции:

5.5.2.5 map_all_files()

```
int map_all_files (
        struct list * list_files )
```

Выделяет всем файлам в списке свой виртуальный адрес.

Аргументы

struct	list *list_files - список файлов

Возвращает

Возвращает в случае успеха вернётся 0, в случае провала -1.

Граф вызовов: Граф вызова функции:

5.5.2.6 processing_all_files()

```
int processing_all_files (
        struct list * list_files )
```

Обрабатывает все файлы: собирает статистику и применяет политику.

Аргументы

struct	list *list_files - список файлов

Возвращает

Возвращает в случае успеха вернётся 0, в случае провала -1.

Граф вызовов: Граф вызова функции:

5.5.2.7 reload_config()

```
int reload_config (
        struct cache_control * cc,
        struct file_monitoring * fm )
```

Перезагружает систему

Аргументы

struct	cache_control *cc - структура управления cache_contorl
struct	file_monitoring *fm - структура файла мониторинга

Возвращает

Возвращает в случае успеха вернётся 0, в случае провала -1.

Функция выполняет следующее:

- Перечитывает файл конфигурации
- Выделяет для файлов новый ударес в виртуальной памяти
- Пересоздаёт файловые дескрипторы

Граф вызовов: Граф вызова функции:

5.5.2.8 start_file_monitoring()

```
void * start_file_monitoring (
        void * args )
```

Запускает процесс мониторинга файлов, используется в соотвествующем потоке

Аргументы

void	args - указатель на struct cache_control

Ждёт, пока epoll обнаружит какое-нибудь событие и вызывает check_type_fd() Граф вызовов:

5.6 File_statistics

Функции

- off_t get_file_size (char *path)
  Позволяет узнать текущий размер файла
- int get_number_of_pages (off_t file_size)
  Позволяет узнать бощее количество страниц, выделенных под файл
- int get_number_of_pages_cache (void *vmem_mapping, off_t size, int number_of_pages)
  Позволяет узнать текущее количество занятых страниц в кэше, выделенных под файл
- int get_percent_cache (int number_of_pages, int number_of_pages_cache)
  Позволяет узнать процентное соотношение занятых страниц по отношению к общему количеству
- int get_last_access (char *path, struct timespec *last_access)
  Позволяет узнать время последнего доступа к файлу
- int get_page_stat (struct file *myfile)
  Позволяет узнать общую информацию о страницах в кэше

5.6.1 Подробное описание

Набор функций, отвечающих за сбор статистики о файле

5.6.2 Функции

5.6.2.1 get_file_size()

```
off_t get_file_size (
            char * path )
```

Позволяет узнать текущий размер файла

Аргументы

char	*path - путь до файла

Возвращает

Возвращает в случае успеха текущий размер файла, в случае ошибки -1

Граф вызова функции:

5.6.2.2 get_last_access()

```
int get_last_access (
            char * path,
            struct timespec * last_access )
```

Позволяет узнать время последнего доступа к файлу

Аргументы

char	*path - путь до файла
struct	timespec *last_access - указатель на структуру времени, куда будет записан результат

Возвращает

Возвращает в случае успеха 0, в случае ошибки -1

Граф вызова функции:

5.6.2.3 get_number_of_pages()

```
int get_number_of_pages (
                off_t file_size )
```

Позволяет узнать бощее количество страниц, выделенных под файл

Аргументы

off↩ _t	file_size - размер файла

Возвращает

Возвращает в случае успеха общее количество страниц, в случае ошибки -1

Граф вызова функции:

5.6.2.4 get_number_of_pages_cache()

```
int get_number_of_pages_cache (
                void * vmem_mapping,
                off_t size,
                int number_of_pages )
```

Позволяет узнать текущее количество занятых страниц в кэше, выделенных под файл

Аргументы

void	*vmem_mapping - указатель на виртуальный адрес
off↩ _t	size - размер файла
int	number_of_pages - общее количество страниц, выделеных под файл

Возвращает

Возвращает в случае успеха текущее количество занятых страниц, в случае ошибки -1

Граф вызова функции:

5.6.2.5 get_page_stat()

```
int get_page_stat (
        struct file * myfile )
```

Позволяет узнать общую информацию о страницах в кэше

Аргументы

struct	file *myfile - указатель на структуру файла, куда будет записан результат

Возвращает

Возвращает в случае успеха 0, в случае ошибки -1

Получает общее количество страниц, количество занятых страниц и их процентное соотношение Граф вызовов: Граф вызова функции:

5.6.2.6 get_percent_cache()

```
int get_percent_cache (
        int number_of_pages,
        int number_of_pages_cache )
```

Позволяет узнать процентное соотношение занятых страниц по отношению к общему количеству

Аргументы

int	number_of_pages - общее количество страниц
int	number_of_pages_cache - количество занятых страниц

Возвращает

Возвращает в случае успеха процент занятых страниц по отношению в общему количетсву

Граф вызова функции:

5.7 Inotify_stuff

Функции

- int add_all_files_to_inotify (int inotify_fd, struct list *file_list)

Добавляет все файлы в inotify список, чтобы тот мог их отслеживать

- int read_inotify (int inotify_fd, struct list *file_list)

  Обрабатывает событие принятое inotify.
- int remove_all_files_from_inotify (int inotify_fd, struct list *file_list)

  Убирает все файлы из inotify списка
- int file_in_attrib (int wd, struct list *file_list)

  Применяет действия в соотвествии с событием IN_ATTRIB.
- int file_in_modify (int wd, struct list *file_list)

  Применяет действия в соотвествии с событием IN_MODIFY.

5.7.1 Подробное описание

Набор функций для работы с inotify

5.7.2 Функции

5.7.2.1 add_all_files_to_inotify()

```
int add_all_files_to_inotify (
            int inotify_fd,
            struct list * file_list )
```

Добавляет все файлы в inotify список, чтобы тот мог их отслеживать

Аргументы

int	inotify_fd - файловый дескриптор inotify
struct	list *file_list - список файлов

Возвращает

Возвращает в случае успеха 0, в случае ошибки -1

Граф вызова функции:

5.7.2.2 file_in_attrib()

```
int file_in_attrib (
            int wd,
            struct list * file_list )
```

Применяет действия в соотвествии с событием IN_ATTRIB.

Аргументы

int	wd - дескриптор просмотра (watch descriptor)
struct	list *file_list - список файлов

Возвращает

Возвращает случае успеха 0, в случае ошибки -1

Если у файла есть виртуальный адрес, пытается получить его размер. Затем если get_file_size() вернёт ошибку, то это значит, что файла больше не существует и он удаляется из списка. В ином случае файлу выделяется новый виртуальный адрес в соотвествии с новым размером. Граф вызовов: Граф вызова функции:

5.7.2.3 file_in_modify()

```
int file_in_modify (
                int wd,
                struct list * file_list )
```

Применяет действия в соотвествии с событием IN_MODIFY.

Аргументы

int	wd - дескриптор просмотра (watch descriptor)
struct	list* *file_list - список файлов

Возвращает

Возвращает в случае успеха 0, в случае ошибки -1

Если у файла есть виртуальный адрес удаляет его и выделяет новый в соотвествии новым размером. Граф вызовов: Граф вызова функции:

5.7.2.4 read_inotify()

```
int read_inotify (
                int inotify_fd,
                struct list * file_list )
```

Обрабатывает событие принятое inotify.

Аргументы

int	inotify_fd - файловый дескриптор inotify
struct	file *file_list - список файлов

Основные события:

- IN_ATTRIB - у файла изменились линки. Вызывается file_in_attrib().
- IN_MOVE_SELF - файл был перемещён. Файл удаляется из списка, так как больше у программы нету способов отслеживать файл.
- IN_MODIFY - файл был изменён. Вызывается file_in_modify().

Граф вызовов: Граф вызова функции:

5.7.2.5 remove_all_files_from_inotify()

```
int remove_all_files_from_inotify (
                int inotify_fd,
                struct list * file_list )
```

Убирает все файлы из inotify списка

Аргументы

int	inotify_fd - файловый дескриптор inotify
struct	list *file_list - список файлов

Возвращает

Возвращает в случае успеха 0, в случае ошибки -1

5.8 Options

Функции

- void print_info ()
  Выводит описание всех доступных опций для ввода в командную строку
- int parse_options (int argc, char **argv, struct cache_control *cc)
  Разбирает аргументы командной строки
- int validate_path_config (char *path)
  Проверяет корректность пути к конфигурационному файлу

5.8.1 Подробное описание

Функции для парсинга аргументов командной строки

5.8.2 Функции

5.8.2.1 parse_options()

```
int parse_options (
                int argc,
                char ** argv,
                struct cache_control * cc )
```

Разбирает аргументы командной строки

Аргументы

int	argc - Количество аргументов командной строки
char	**argv - Массив аргументов командной строки
struct	cache_control *cc - Указатель на структуру cache_control для сохранения результата

Возвращает

Возвращает 0 в случае успеха, -1 в случае ошибки или -2 в случае вызова меню с описанием опций

Обрабатывает флаги -d, -c, -h. Если путь к конфигу не указан, использует путь по умолчанию config/cachecontrol.conf, перед завершением проверяет корректность пути к конфигурационному файлу Граф вызовов:

5.8.2.2 print_info()

void print_info ( )

Выводит описание всех доступных опций для ввода в командную строку

Опции программы: 1) -d - Запуск в фоновом режиме (запускается как демон) 2) -c <config> - Запуск с указанием пути к конфигурационному файлу (запускается не в фоне) 3) -d -c - Запуск с указанием пути к конфигурационному файлу (запускается как демон) 4) никаких опций - Запуск не в фоновом режиме 5) -h - Выводит меню с описанием всех опций Граф вызова функции:

5.8.2.3 validate_path_config()

int validate_path_config (
char * path)

Проверяет корректность пути к конфигурационному файлу

Аргументы

char	*path - путь к конфигурационному файлу

Возвращает

Возвращает 0 в случае успеха или -1 в случае ошибки

Использует stat() для проверки существования пути Граф вызова функции:

5.9 Parse_config

Модуль парсинга конфигурационного файла кэширования

Файлы

- файл parse_config.h

  Заголовочный файл парсера конфигурации кэширования

Макросы

- #define DEFAULT_INTERVAL 60
- #define SIZE_BUFF 4096

Функции

- int parse_config (const char *config_path, struct list *l, int *interval)
  Разбирает конфигурационный файл и заполняет список файлов

5.9.1 Подробное описание

Модуль парсинга конфигурационного файла кэширования

Предоставляет функции для разбора конфигурационного файла, содержащего правила управления кэшем для файлов и каталогов. Поддерживает три политики: keep, evict, monitor.

5.9.2 Макросы

5.9.2.1 DEFAULT_INTERVAL

#define DEFAULT_INTERVAL 60

Значение интервала обновления по умолчанию (в секундах)

5.9.2.2 SIZE_BUFF

#define SIZE_BUFF 4096

Размер буфера для чтения строк конфигурационного файла

5.9.3 Функции

5.9.3.1 parse_config()

```
int parse_config (
            const char * config_path,
            struct list * l,
            int * interval )
```

Разбирает конфигурационный файл и заполняет список файлов

Читает конфигурационный файл построчно, парсит правила для файлов и каталогов. Поддерживаются три политики: keep (постоянно держать в кэше), evict (выгружать через age секунд), monitor (только статистика).

Формат строки конфигурации: <путь_с_маской> <политика> [age=<секунды>]

Примеры:

```
/var/lib/mysql/* keep
/var/log/*.log evict age=3600
/tmp/* evict age=300
/opt/data/*.dat monitor
```

Аргументы

in	config_path	Путь к конфигурационному файлу
out	l	Список, в который добавляются структуры file
out	interval	Значение интервала обновления (в секундах)

Возвращает

- 0 - успех
- 1 - ошибка открытия файла
- 2 - синтаксическая ошибка
- 3 - семантическая ошибка (разные политики для одного файла)

Граф вызова функции:

5.10 Signal_handler

Функции

- void sigint_handler (int sig)
  Обработчик сигнала SIGINT.
- void sigterm_handler (int sig)
  Обработчик сигнала SIGTERM.
- void sighup_handler (int sig)
  Обработчик сигнала SIGHUP.
- void sigusr1_handler (int sig)
  Обработчик сигнала SIGUSR1.
- int register_handler (int signum, void(*handler)(int))
  Регистрирует обработчик для указанного сигнала
- void init_signal_handler ()
  Инициализирует все обработчики сигналов программы
- void set_context (struct cache_control *cc)
  Устанавливает контекст программы (cache_control) для обработчиков сигналов

5.10.1 Подробное описание

Функции для обработки системных сигналов

5.10.2 Функции

5.10.2.1 init_signal_handler()

void init_signal_handler ( )

Инициализирует все обработчики сигналов программы

Регистрирует обработчики для сигналов:

- SIGINT и SIGTERM — завершение работы (shutdown_flag)
- SIGHUP — перезагрузка конфигурации (reload_flag)
- SIGUSR1 — запрос статистики (stats_flag)

Граф вызовов:

5.10.2.2 register_handler()

```
int register_handler (
            int signum,
            void(*)(int) handler )
```

Регистрирует обработчик для указанного сигнала

Аргументы

int	signum - номер сигнала (например, SIGINT, SIGTERM)
void	(*handler)(int) - указатель на функцию-обработчик

Возвращает

Возвращает 0 в случае успеха или -1 в случае ошибки

Использует sigaction() для установки обработчика сигнала Граф вызова функции:

5.10.2.3 set_context()

```
void set_context (
            struct cache_control * cc )
```

Устанавливает контекст программы (cache_control) для обработчиков сигналов

Аргументы

struct	cache_control *cc - указатель на контекст программы

Сохраняет указатель на контекст, чтобы обработчики сигналов могли изменять соответствующие флаги

5.10.2.4 sighup_handler()

```
void sighup_handler (
            int sig )
```

Обработчик сигнала SIGHUP.

Аргументы

int	sig - номер принятого сигнала

Устанавливает флаг reload_flag в контексте программы (cache_control) Граф вызова функции:

5.10.2.5 sigint_handler()

void sigint_handler (
int sig)

Обработчик сигнала SIGINT.

Аргументы

int	sig - номер принятого сигнала

Устанавливает флаг shutdown_flag в контексте программы(cache_control) Граф вызова функции:

5.10.2.6 sigterm_handler()

void sigterm_handler (
int sig)

Обработчик сигнала SIGTERM.

Аргументы

int	sig - номер принятого сигнала

Устанавливает флаг shutdown_flag в контексте программы (cache_control) Граф вызова функции:

5.10.2.7 sigusr1_handler()

void sigusr1_handler (
int sig)

Обработчик сигнала SIGUSR1.

Аргументы

int	sig - номер принятого сигнала

Устанавливает флаг stats_flag в контексте программы (cache_control) Граф вызова функции:

5.11 Threads

Функции

- int start_threads (struct cache_control *cc)
  Запускает рабочие потоки программы
- int stop_threads ()
  Останавливает все рабочие потоки программы

5.11.1 Подробное описание

Функции для управления рабочими потоками программы

5.11.2 Функции

5.11.2.1 start_threads()

int start_threads (
struct cache_control * cc)

Запускает рабочие потоки программы

Аргументы

struct	cache_control *cc - указатель на контекст программы

Возвращает

Возвращает 0 в случае успеха или -1 в случае ошибки

Создаёт два потока:

- поток мониторинга файлов (file_monitoring_thread)
- поток обработки команд из FIFO (command_listener_thread)

5.11.2.2 stop_threads()

int stop_threads ( )

Останавливает все рабочие потоки программы

Возвращает

Возвращает 0 в случае успеха или -1 в случае ошибки

Оожидает завершения потоков мониторинга и обработчика команд через pthread_join().

5.12 control mediator

Модуль управления кэшем через очередь сообщений

Структуры данных

- struct command_t

  Структура, представляющая команду управления кэшем
- struct file_info

  Структура, которая содержит статистику о файле
- struct reply_t

  Структура, которая содержит ответ от демона на запрос медиатора
- struct mqfd_t

  Структура для хранения дескрипторов очередей сообщений

Макросы

- #define SEND_FILE "/snd_queue"
- #define RECV_FILE "/rcv_queue"
- #define CLEAR_TERMINAL "\033[2J\033[H"
- #define RED "\033[31m"
- #define DASHES "----------\n"
- #define N 255
- #define MAX_TOK 2
- #define PATH_LEN 255
- #define COMMAND_LEN 10
- #define PATH_BUFF_SIZE 255
- #define PATH_LOG_FILE "~/logs/cache_control_mediator.log"
- #define POLITIC_LEN 15

Перечисления

- enum pol { P_KEEP = 1 , P_EVICT , P_MONITOR }

  Политики кэширования файлов
- enum command_type_t {
  CMD_EXIT , CMD_RELOAD , CMD_EVICT , CMD_CACHE ,
  CMD_STATUS , CMD_CLEAR , CMD_UNKNOWN }

  Команды, поддерживаемые медиатором

Функции

- int read_string (char *str)

  Считывает строку из стандартного ввода
- void clear_stdin ()

  Очищает буфер стандартного ввода
- void print_menu ()

  Выводит меню доступных команд
- int get_command (char *str, command_t *command, command_type_t *op)

  Разбирает введённую строку на команду и её аргументы
- int print_answer (const char *const answer, const command_t *const ptr_command, const command_type_t op)

  Выводит ответ от демона в удобочитаемом формате
- int split_tokens (char *str, char *tokens[], const char *delim, size_t *count_tok)

  Разбивает строку на токены по разделителю

- mqfd_t open_queue (const char *send_path, const char *recv_path, const int send_flags, const int recv_flags)

  Открывает очереди сообщений для обмена с демоном
- int set_command (const char *str, command_t *command, const char *path, const command_type_t op)

  Формирует структуру команды из строки
- command_type_t get_command_type (const char *cmd)

  Определяет тип команды по её имени
- int get_politic_str (reply_t *replay, char *str)

  Преобразует числовой код политики в строку

5.12.1 Подробное описание

Модуль управления кэшем через очередь сообщений

Предоставляет интерфейс для взаимодействия с демоном кэширования через очереди сообщений POSIX. Позволяет отправлять команды управления кэшем и получать статистику по файлам.

5.12.2 Макросы

5.12.2.1 CLEAR_TERMINAL

#define CLEAR_TERMINAL "\033[2J\033[H"

Управляющая последовательность для очистки терминала

5.12.2.2 COMMAND_LEN

#define COMMAND_LEN 10

Максимальная длина имени команды

5.12.2.3 MAX_TOK

#define MAX_TOK 2

Максимальное количество токенов при разборе командной строки

5.12.2.4 N

#define N 255

Максимальный размер буфера для ввода команд

5.12.2.5 PATH_BUFF_SIZE

#define PATH_BUFF_SIZE 255

Размер буфера для хранения пути

5.12.2.6 PATH_LEN

#define PATH_LEN 255

Максимальная длина пути к файлу

5.12.2.7 PATH_LOG_FILE

#define PATH_LOG_FILE "~/logs/cache_control_mediator.log"

Путь к файлу для логирования

5.12.2.8 POLITIC_LEN

#define POLITIC_LEN 15

Максимальная длина названия политики кэширования

5.12.2.9 RECV_FILE

#define RECV_FILE "/rcv_queue"

Путь к очереди для приёма сообщений от демона

5.12.2.10 RED

#define RED "\033[31m"

Цвет текста (красный) для вывода ошибок

5.12.2.11 SEND_FILE

#define SEND_FILE "/snd_queue"

Путь к очереди для отправки сообщений демону

5.12.3 Перечисления

5.12.3.1 command_type_t

enum command_type_t

Команды, поддерживаемые медиатором

Используется для идентификации команды при обработке

Элементы перечислений

CMD_RELOAD	Команда завершения работы демона
CMD_EVICT	Команда перезагрузки конфигурации
CMD_CACHE	Команда принудительной выгрузки файла
CMD_STATUS	Команда принудительной загрузки файла в кэш
CMD_CLEAR	Команда получения статистики по файлу
CMD_UNKNOWN	Команда очистки терминала Неизвестная команда

5.12.3.2 pol

enum pol

Политики кэширования файлов

Определяют поведение демона при управлении кэшем файлов

Элементы перечислений

P_KEEP	Постоянно держать файл в кэше
P_EVICT	Выгружать файл, если им не пользовались age секунд
P_MONITOR	Только собирать статистику, не влияя на кэш

5.12.4 Функции

5.12.4.1 clear_stdin()

void clear_stdin ( )

Очищает буфер стандартного ввода

Считывает и отбрасывает символы до конца строки или EOF Граф вызова функции:

5.12.4.2 get_command()

int get_command (
char * str,
command_t * command,
command_type_t * op)

Разбирает введённую строку на команду и её аргументы

Аргументы

in	str	Входная строка для разбора
out	command	Структура для заполнения командой и путём
out	op	Тип команды

Возвращает

0 - успех, 1 - ошибка синтаксиса, 2 - команда clear

Граф вызовов:

5.12.4.3 get_command_type()

```
command_type_t get_command_type (
        const char * cmd )
```

Определяет тип команды по её имени

Аргументы

in	cmd	Строка с именем команды

Возвращает

Тип команды (command_type_t)

Граф вызова функции:

5.12.4.4 get_politic_str()

```
int get_politic_str (
        reply_t * replay,
        char * str )
```

Преобразует числовой код политики в строку

Аргументы

in	replay	Указатель на структуру ответа с политикой
out	str	Буфер для записи названия политики

Возвращает

0 - успех, 1 - неизвестная политика

Граф вызова функции:

5.12.4.5 open_queue()

```
mqfd_t open_queue (
        const char * send_path,
        const char * recv_path,
        const int send_flags,
        const int recv_flags )
```

Открывает очереди сообщений для обмена с демоном

Аргументы

in	send_path	Путь к очереди для отправки сообщений
in	recv_path	Путь к очереди для приёма сообщений
in	send_flags	Флаги открытия очереди отправки
in	recv_flags	Флаги открытия очереди приёма

Возвращает

Структура с дескрипторами открытых очередей

5.12.4.6 print_answer()

```
int print_answer (
        const char *const answer,
        const command_t *const ptr_command,
        const command_type_t op )
```

Выводит ответ от демона в удобочитаемом формате

Аргументы

in	answer	Буфер с ответом от демона
in	ptr_command	Указатель на отправленную команду
in	op	Тип выполненной команды

Возвращает

0 - успех, 1 - ошибка открытия файла, 2 - файл не найден, 3 - ошибка определения политики

Граф вызовов:

5.12.4.7 print_menu()

```
void print_menu ( )
```

Выводит меню доступных команд

Отображает список всех поддерживаемых команд и их назначение

5.12.4.8 read_string()

```
int read_string (
        char * str )
```

Считывает строку из стандартного ввода

Аргументы

out	str	- Буфер для хранения считанной строки

Возвращает

0 - успех, 1 - ошибка ввода, 2 - строка слишком длинная

Граф вызовов:

5.12.4.9 set_command()

```
int set_command (
        const char * str,
        command_t * command,
        const char * path,
        const command_type_t op )
```

Формирует структуру команды из строки

Аргументы

in	str	Имя команды
out	command	Структура для заполнения
in	path	Путь к файлу (или пустая строка)
in	op	Тип команды

Возвращает

0 - успех, 1 - ошибка

Граф вызова функции:

5.12.4.10 split_tokens()

```
int split_tokens (
        char * str,
        char * tokens[],
        const char * delim,
        size_t * count_tok )
```

Разбивает строку на токены по разделителю

Аргументы

in	str	Входная строка для разбиения
out	tokens	Массив указателей на токены
in	delim	Строка-разделитель
out	count_tok	Количество полученных токенов

Возвращает

0 - успех, 1 - превышено максимальное количество токенов

Граф вызова функции:

5.13 List

Структуры данных

- struct file_stat
- struct file
- struct node
- struct list

Макросы

- #define PATH_BUFF_SIZE 255 /* размер буфера пути файла */

Перечисления

- enum politics { KEEP = 1 , EVICT , MONITOR }

Функции

- struct file_stat * init_file_stat (off_t s, int np, int npc, int pc, struct timespec *la)

  Функция инициализации структуры статистики файла
- struct file_stat * init_empty_stat ()

  Функция инициализации пустой структуры статистики файла \retrun Возвращает казатель на структуру файла
- struct file * init_file (char *pb, int p, int a)

  Функция инициализации структуры файла
- int add_file_stat (struct file *f, struct file_stat *fs)

  Добавляет стутс к списку
- struct node * init_node (struct file *f)

  Функция инициализации вершины
- struct list * init_list ()

  Функция инициализации списка
- int add_file (struct list *l, struct file *f)

  Добавляет файл в список
- struct file * find_file_by_path (struct list *l, char *path)

  Находит файл по его пути
- struct file * find_file_by_wd (struct list *l, int wd)

  Находит файл в списке по его дескриптору просмотра (watch descriptor)
- int delete_file_by_path (struct list *l, char *path)

  Удаляет файл из списка по его пути
- int delete_file_by_wd (struct list *l, int wd)

  Удаляет файл из списка по его дескриптору просмотра (watch descriptor)
- int free_list (struct list *l)

  Высвобождает память, выделенную под список

5.13.1 Подробное описание

Набор функций для списка файлов

5.13.2 Перечисления

5.13.2.1 politics

enum politics

Основные политики файла:

- KEEP - постоянно держит файл в кэше
- EVICT - выгружает файл, если им не пользовались age секунд
- MONITOR - собирает статистику

5.13.3 Функции

5.13.3.1 add_file()

int add_file (
struct list * l,
struct file * f)

Добавляет файл в список

Аргументы

struct	list *l - список, куда добавится файл
struct	file *f - файл, который добавится в список

Возвращает

В случае успеха 0, в случае ошибки -1

Граф вызовов:

5.13.3.2 add_file_stat()

int add_file_stat (
struct file * f,
struct file_stat * fs)

Добавляет стутс к списку

Аргументы

struct	file *f - структура файла
struct	file_stat *fs - структура статистики файла

Возвращает

В случае успеха 0, в случае ошибки -1

5.13.3.3 delete_file_by_path()

```
int delete_file_by_path (
                struct list * l,
                char * path )
```

Удаляет файл из списка по его пути

Аргументы

struct	list *l - список, откуда будет удалён файл
char	*path - путь до файла

Возвращает

В случае успеха 0, в случае ошибки -1

5.13.3.4 delete_file_by_wd()

```
int delete_file_by_wd (
                struct list * l,
                int wd )
```

Удаляет файл из списка по его дескриптору просмотра (watch descriptor)

Аргументы

struct	list *l - список, откуда будет удалён файл
int	wd - дескриптор просмотра

Возвращает

В случае успеха 0, в случае ошибки -1

Граф вызова функции:

5.13.3.5 find_file_by_path()

```
struct file * find_file_by_path (
                struct list * l,
                char * path )
```

Находит файл по его пути

Аргументы

struct	list *l - список, где будет производится поиск
char	*path - путь до файла

Возвращает

Возвращает в случае успеха указатель на файл, в случае ошибки -1

5.13.3.6 find_file_by_wd()

```
struct file * find_file_by_wd (
                struct list * l,
                int wd )
```

Находит файл в списке по его дескриптору просмотра (watch descriptor)

Аргументы

struct	list *l - список, где будет производится поиск
int	wd - дескриптор просмотра

Возвращает

Возвращает в случае успеха указатель на файл, в случае ошибки -1

Граф вызова функции:

5.13.3.7 free_list()

```
int free_list (
                struct list * l )
```

Высвобождает память, выделенную под список

Аргументы

struct	list *l - список, который будет удалён

Возвращает

В случае успеха 0, в случае ошибки -1

Граф вызова функции:

5.13.3.8 init_empty_stat()

```
struct file_stat * init_empty_stat ( )
```

Функция инициализации пустой структуры статистики файла \retrun Возвращает казатель на структуру файла

Предупреждения

Память для статистики выделяется денамически, поэтому после использования рекомендуется вызвать функцию free()

Граф вызова функции:

5.13.3.9 init_file()

```
struct file * init_file (
            char * pb,
            int p,
            int a )
```

Функция инициализации структуры файла

Аргументы

char	*pb - путь файла
int	p - политика
int	a - время через, которое файл будет выгружен при политике EVICT

Возвращает

Возвращает указатель на структуру файла

Предупреждения

Память для структуры файла выделяется денамически, поэтому после использования рекомендуется вызвать функцию free()

5.13.3.10 init_file_stat()

```
struct file_stat * init_file_stat (
            off_t s,
```

```
                int np,
                int npc,
                int pc,
                struct timespec * la )
```

Функция инициализации структуры статистики файла

Аргументы

off↩ _t	s - размер файла
int	np - общее количество страниц
int	npc - количество занятых страниц
int	pc - процентное соотщение занятых страниц к общему количеству
struct	timespec *la - время последнего доступа к файлу \retrun Возвращает указатель на структуру файла

Предупреждения

Память для структуры статистики выделяется динамически, поэтому после использования рекомендуется вызвать функцию free()

5.13.3.11 init_list()

```
struct list * init_list ( )
```

Функция инициализации списка

Возвращает

Возвращает указатель на список

Предупреждения

Память для списка выделяется денамически, поэтому после использования рекомендуется вызвать функцию free_list();

Граф вызова функции:

5.13.3.12 init_node()

```
struct node * init_node (
                struct file * f )
```

Функция инициализации вершины

Аргументы

struct	file *f - файл, который будет добавлен в вершину

Предупреждения

Память для вершины выделяется денамически, поэтому после использования рекомендуется вызвать функцию free()

Граф вызова функции:

5.14 Queue

Макросы

- #define SIZE_RCV 102
- #define SIZE_SND 4
- #define MAX_SIZE_MSG 8192
- #define CHANGE_SIZE_MSG 8190
- #define MQ_TIMEOUT 5

Функции

- mqd_t create_queue (char *file_desc, int flags, int size_msg)
  Функция создания.присоеденения к очереди сообщений
- char * recive_msg (mqd_t queue, char *snd_msg, int size_msg)
  Функция приема сообщений из очереди
- int send_msg (mqd_t queue, char *snd_msg, int size_msg)
  Функция отправки сообщения в очередь
- char * recive_msg_for_mediator (mqd_t queue_fd, char *rcv_msg, int size_msg)
  Функция отправки сообщения в очередь для mediator.

5.14.1 Подробное описание

Набор функций для взаимодействия с очередями сообщений между command listener и mediator

5.14.2 Макросы

5.14.2.1 CHANGE_SIZE_MSG

#define CHANGE_SIZE_MSG 8190

размер сообщения для очередей если был превышено максимальное значение

5.14.2.2 MAX_SIZE_MSG

#define MAX_SIZE_MSG 8192

максимальновозможный размер сообщения для очередей

5.14.2.3 MQ_TIMEOUT

#define MQ_TIMEOUT 5

Таймаут ожидания сообщения из очереди

5.14.2.4 SIZE_RCV

#define SIZE_RCV 102

размер сообщения для передачи

5.14.2.5 SIZE_SND

#define SIZE_SND 4

размер сообщения для приема

5.14.3 Функции

5.14.3.1 create_queue()

```
mqd_t create_queue (
                char * file_desc,
                int flags,
                int size_msg )
```

Функция создания.присоеденения к очереди сообщений

Аргументы

char	*file_desc - уникальное имя идентификатор для очереди
int	flags - принимаемые флаги определяющие права для очереди O_RDONLY/O_WRONLY и флаг для создания новой O_CREAT
int	size_msg - размер сообщения которые будут передаваться в очереди

Возвращает

возвращает идентификатор очереди

5.14.3.2 recive_msg()

```
char * recive_msg (
                mqd_t queue,
                char * snd_msg,
                int size_msg )
```

Функция приема сообщений из очереди

Аргументы

mqd↩ _t	queue - идентификатор очереди откуда будут приниматься сообщений
char	∗snd_msg - буфер куда бедет записываться сообщение из очереди
int	size_msg - размер сообщения которое будет вычитываться из очереди

Возвращает

возвращает буфер с полученным сообщением. В случае ошибки возвращает NULL

5.14.3.3 recive_msg_for_mediator()

```
char ∗ recive_msg_for_mediator (
                mqd_t queue_fd,
                char ∗ rcv_msg,
                int size_msg )
```

Функция отправки сообщения в очередь для mediator.

Аргументы

mqd↩ _t	queue - идентификатор очереди откуда будут приниматься сообщений
char	∗snd_msg - сообщение которое будет переданно в очередь
int	size_msg - размер сообщения которое будет передаваться в очередь

Предупреждения

отличается от функции recive_msg тем что в ней нет таймера для прерывания ожидания сообщения

5.14.3.4 send_msg()

```
int send_msg (
                mqd_t queue,
                char ∗ snd_msg,
                int size_msg )
```

Функция отправки сообщения в очередь

Аргументы

mqd↩ _t	queue - идентификатор очереди откуда будут приниматься сообщений
char	∗snd_msg - сообщение которое будет переданно в очередь
int	size_msg - размер сообщения которое будет передаваться в очередь

Глава 6

Структуры данных

6.1 Структура cache_control

#include <context.h>

Поля данных

- int daemon_flag
- char config_path [PATH_SIZE]
- struct list * file_list
- int shutdown_flag
- int interval
- int reload_flag
- int stats_flag
- pthread_mutex_t mutex

6.1.1 Подробное описание

Содержит главную информацию о состоянии программы cache_control

Объявления и описания членов структуры находятся в файле:

- include/cache_control/context.h

6.2 Структура command_t

Структура, представляющая команду управления кэшем

#include <cache_control_mediator.h>

Поля данных

- char command [COMMAND_LEN]
- char path_to_file [PATH_LEN]

6.2.1 Подробное описание

Структура, представляющая команду управления кэшем

Содержит имя команды и путь к файлу (если требуется)

6.2.2 Поля

6.2.2.1 command

char command[COMMAND_LEN]

Имя команды (status, cache, evict и т.д.)

6.2.2.2 path_to_file

char path_to_file[PATH_LEN]

Путь к файлу (для команд, требующих аргумент)

Объявления и описания членов структуры находятся в файле:

- include/cache_control_mediator/cache_control_mediator.h

6.3 Структура file

#include <list.h>

Поля данных

- char path [PATH_BUFF_SIZE]
- struct file_stat * stat
- int politic
- int age
- int wd
- int fd
- void * vmem_mapping

6.3.1 Подробное описание

Хранит основную информацию о файле

Объявления и описания членов структуры находятся в файле:

- include/list/list.h

6.4 Структура file_info

Структура, которая содержит статистику о файле

#include <command_listener.h>

Поля данных

- char path [PATH_BUFF_SIZE]
- off_t size
- int number_of_pages
- int number_of_pages_cache
- int percent_cache
- struct timespec last_access
- int politic
- int age

6.4.1 Подробное описание

Структура, которая содержит статистику о файле

Структура в которую будут записывается информация о файле для передачи mediator

6.4.2 Поля

6.4.2.1 age

int age

время через которое файл будет выгружен из памяти

6.4.2.2 last_access

struct timespec last_access

время последнего обращения к файлу

Процентное соотношение занятых страниц к общему количеству

6.4.2.3 number_of_pages

int number_of_pages

общее количество страниц файлакоторые можно подгрузить в памть

Рамер файла в байтах

6.4.2.4 number_of_pages_cache

int number_of_pages_cache

количество страниц файла которые уже загружены в память

Общее количество страниц

6.4.2.5 path

char path

путь до файла который указан в конфигурациооном файле

Путь к файлу

6.4.2.6 percent_cache

int percent_cache

процент страниц загруженных страниц файла в память

Количество занятых страниц

6.4.2.7 politic

int politic

политика для маниторинга файла в памяти

Время последнего доступа к файлу

6.4.2.8 size

off_t size

размер файла

Объявления и описания членов структур находятся в файлах:

- include/cache_control/command_listener.h
- include/cache_control_mediator/cache_control_mediator.h

6.5 Структура file_monitoring

#include <file_monitoring.h>

Поля данных

- int epoll_fd
- int main_timer_fd
- int inotify_fd

6.5.1 Подробное описание

Содержит набор файловых дескрипторов, необходимых для мониторинга файлов

Объявления и описания членов структуры находятся в файле:

- include/cache_control/file_monitoring.h

6.6 Структура file_stat

#include <list.h>

Поля данных

- off_t size
- int number_of_pages
- int number_of_pages_cache
- int percent_cache
- struct timespec last_access

6.6.1 Подробное описание

Хранит основную информацию о статистики файла

Объявления и описания членов структуры находятся в файле:

- include/list/list.h

6.7 Структура list

#include <list.h>

Поля данных

- struct node * start
- int length
- int evict_files

6.7.1 Подробное описание

Хранит основную информацию о списке

Объявления и описания членов структуры находятся в файле:

- include/list/list.h

6.8 Структура mediator_command

Поля данных

- char com [COMMAND_LEN]
- char path_to_file [PATH_FILE]

6.8.1 Поля

6.8.1.1 com

char com[COMMAND_LEN]

команда от mediator

6.8.1.2 path_to_file

char path_to_file[PATH_FILE]

аргумент для команды. Может быть пустым для некоторых команд

Объявления и описания членов структуры находятся в файле:

- include/cache_control/command_listener.h

6.9 Структура mqfd_t

Структура для хранения дескрипторов очередей сообщений

#include <cache_control_mediator.h>

Поля данных

- mqd_t fds [2]

6.9.1 Подробное описание

Структура для хранения дескрипторов очередей сообщений

Содержит два дескриптора: для отправки и приёма сообщений

6.9.2 Поля

6.9.2.1 fds

mqd_t fds[2]

Массив дескрипторов: [0] - приём, [1] - отправка

Объявления и описания членов структуры находятся в файле:

- include/cache_control_mediator/cache_control_mediator.h

6.10 Структура node

#include <list.h>

Поля данных

- struct file * nd_file
- struct node * nd_next

6.10.1 Подробное описание

Хранит основную информацию о вершине списка

Объявления и описания членов структуры находятся в файле:

- include/list/list.h

6.11 Структура replay_t

#include <command_listener.h>

Поля данных

- int status
- int last_msg
- struct file_info f

6.11.1 Подробное описание

структура для передачи mediator

6.11.2 Поля

6.11.2.1 f

struct file_info f

структура с информацией о файле

6.11.2.2 last_msg

int last_msg

обозначение последнего сообщения в передаче сообщений для mediator

6.11.2.3 status

int status

информационное поле для обозначения принята команда в обработку или получена ошибка

Объявления и описания членов структуры находятся в файле:

- include/cache_control/command_listener.h

6.12 Структура reply_t

Структура, которая содержит ответ от демона на запрос медиатора

#include <cache_control_mediator.h>

Поля данных

- int status
- int last_msg
- struct file_info f

6.12.1 Подробное описание

Структура, которая содержит ответ от демона на запрос медиатора

Содержит статус выполнения и статистику по файлу

Объявления и описания членов структуры находятся в файле:

- include/cache_control_mediator/cache_control_mediator.h

Глава 7

Файлы

7.1 command_listener.h

```
00001 #ifndef CACHECONTROL_LISTNER_H
00002 #define CACHECONTROL_LISTNER_H
00003
00004 #include "file_loading.h"
00005 #include "context.h"
00006 #include "../queue/queue.h"
00007 #include "../log/log.h"
00008 #include "file_statistics.h"
00009 #include "file_monitoring.h"
00010 #include "../list/list.h"
00011
00016 #define COMMAND_LEN 10
00017 #define PATH_FILE 255
00021 struct file_info {
00022     char path[PATH_BUFF_SIZE];
00024     off_t size;
00025     int number_of_pages;
00026     int number_of_pages_cache;
00027     int percent_cache;
00028     struct timespec last_access;
00030     int politic;
00031     int age;
00032 };
00033
00037 struct replay_t {
00038     int status;
00039     int last_msg;
00040     struct file_info f;
00041 };
00042 /* структура для приема сообщения от mediator с командой*/
00043 struct mediator_command {
00044     char com[COMMAND_LEN];
00045     char path_to_file
00046         [PATH_FILE];
00047 };
00056 int status_all(struct node *mynode, struct replay_t snd_msg, mqd_t snd_queue,
00057             struct cache_control *cc);
00065 int status_spec(struct file *myfile, struct replay_t snd_msg, mqd_t snd_queue,
00066         struct cache_control *cc);
00073 int do_cache(struct file *myfile, struct replay_t snd_msg, mqd_t snd_queue);
00082 int do_evict(struct file *myfile, struct replay_t snd_msg, mqd_t snd_queue,
00083          struct cache_control *cc);
00090 int do_reload(struct cache_control *cc, struct replay_t snd_msg,
00091           mqd_t snd_queue);
00099 int do_exit(struct cache_control *cc, struct replay_t snd_msg, mqd_t rcv_queue,
00100         mqd_t snd_queue);
00105 int command_listner(void *args);
00107 #endif
```

7.2 context.h

```
00001 #pragma once
00002
00003 #ifndef CONTEXT_H
```

```
00004 #define CONTEXT_H
00005
00006 #include <pthread.h>
00007 #include "../list/list.h"
00008 #include "parse_config.h"
00009 #include "../log/log.h"
00010
00016 #define PATH_SIZE 255 /* Максимальный размер пути к конфигурационному файлу */
00017
00019 struct cache_control {
00020     int daemon_flag; /* флаг запуска в режиме демона */
00021     char config_path[PATH_SIZE]; /* путь к конфигурационному файлу */
00022     struct list *file_list; /* список отслеживаемых файлов */
00023     int shutdown_flag; /* флаг завершения работы программы */
00024     int interval; /* интервал мониторинга в секундах */
00025     int reload_flag; /* флаг перезагрузки конфигурации */
00026     int stats_flag; /* флаг запроса статистики */
00027     pthread_mutex_t mutex; /* мьютекс для синхронизации доступа к контексту*/
00028 };
00029
00036 int init_context(struct cache_control *cc);
00037
00044 int remove_context(struct cache_control *cc);
00045
00048 #endif
```

7.3 daemonize.h

```
00001 #pragma once
00002
00003 #ifndef DAEMONIZE_H
00004 #define DAEMONIZE_H
00005
00006 #include <stdio.h>
00007 #include <unistd.h>
00008 #include "../log/log.h"
00009
00021 int daemonize();
00022
00025 #endif
```

7.4 events_stuff.h

```
00001 #pragma once
00002
00003 #ifndef EVENTS_STUFF_H
00004 #define EVENTS_STUFF_H
00005
00006 #include <sys/timerfd.h>
00007 #include <sys/epoll.h>
00008 #include <unistd.h>
00009 #include <stdio.h>
00010 #include "../log/log.h"
00011
00017 #define TIMEOUT 1000  /* время, через которое epoll перестанет ждать сигнал */
00018
00019 #define MAX_NUM_OF_EVENTS 2 /* количество событий, которое может произойти одновремено */
00020
00027 int set_timer(int timer_fd, time_t seconds);
00028
00035 int add_fd_to_epoll(int epoll_fd, int fd);
00036
00039 #endif
```

7.5 file_loading.h

```
00001 #pragma once
00002
00003 #ifndef FILE_LOADING_H
00004 #define FILE_LOADING_H
00005
00006 #include <stdio.h>
00007 #include <unistd.h>
00008 #include <sys/mman.h>
00009 #include <errno.h>
00010 #include <fcntl.h>
```

```
00011 #include "../list/list.h"
00012 #include "../log/log.h"
00013
00025 int map_file(struct file *myfile);
00026
00032 int load_file(struct file *myfile);
00033
00039 int unmap_file(struct file *myfile);
00040
00042 #endif
```

7.6 file_monitoring.h

```
00001 #pragma once
00002
00003 #ifndef FILE_MONITORING_H
00004 #define FILE_MONITORING_H
00005
00006 #include <stdio.h>
00007 #include <stdlib.h>
00008 #include <sys/epoll.h>
00009 #include <sys/timerfd.h>
00010 #include <unistd.h>
00011 #include <sys/inotify.h>
00012
00013 #include "context.h"
00014 #include "../list/list.h"
00015 #include "file_statistics.h"
00016 #include "file_loading.h"
00017 #include "events_stuff.h"
00018 #include "inotify_stuff.h"
00019
00025 #define MIN_PERCENT_CACHE 80 /* минимальный процент кэша для политики EVICT */
00026
00028 struct file_monitoring {
00029     int epoll_fd; /* файловый дескриптор epoll */
00030     int main_timer_fd; /* файловый дескриптор таймера */
00031     int inotify_fd; /* файловый дескриптор inotify */
00032 };
00033
00040 int init_file_monitoring(struct file_monitoring *fm);
00041
00047 void *start_file_monitoring(void *args);
00048
00058 int do_file_statistics(struct file *myfile);
00059
00069 int do_file_loading(struct file *myfile);
00070
00076 int map_all_files(struct list *list_files);
00077
00083 int processing_all_files(struct list *list_files);
00084
00094 int check_type_fd(int fd, struct file_monitoring *fm, struct list *file_list);
00095 int reload_config(struct cache_control *cc, struct file_monitoring *fm);
00096
00107 int reload_config(struct cache_control *cc, struct file_monitoring *fm);
00108
00111 #endif
```

7.7 file_statistics.h

```
00001 #pragma once
00002
00003 #ifndef FILE_STATISTICS_H
00004 #define FILE_STATISTICS_H
00005
00006 #include <stdint.h>
00007 #include <stdio.h>
00008 #include <unistd.h>
00009 #include <sys/mman.h>
00010 #include <sys/stat.h>
00011 #include <errno.h>
00012 #include <fcntl.h>
00013 #include "../list/list.h"
00014 #include "../log/log.h"
00015
00026 off_t get_file_size(char *path);
00027
00033 int get_number_of_pages(off_t file_size);
```

```
00034
00042 int get_number_of_pages_cache(void *vmem_mapping, off_t size,
00043                 int number_of_pages);
00044
00051 int get_percent_cache(int number_of_pages, int number_of_pages_cache);
00052
00059 int get_last_access(char *path, struct timespec *last_access);
00060
00067 int get_page_stat(struct file *myfile);
00068
00071 #endif
```

7.8 inotify_stuff.h

```
00001 #pragma once
00002
00003 #ifndef INOTIFY_STUFF_H
00004 #define INOTIFY_STUFF_H
00005
00006 #include <sys/inotify.h>
00007 #include <unistd.h>
00008 #include <string.h>
00009 #include <stdio.h>
00010 #include "../list/list.h"
00011 #include "file_loading.h"
00012 #include "file_statistics.h"
00013
00025 int add_all_files_to_inotify(int inotify_fd, struct list *file_list);
00026
00036 int read_inotify(int inotify_fd, struct list *file_list);
00037
00044 int remove_all_files_from_inotify(int inotify_fd, struct list *file_list);
00045
00055 int file_in_attrib(int wd, struct list *file_list);
00056
00064 int file_in_modify(int wd, struct list *file_list);
00065
00067 #endif
```

7.9 options.h

```
00001 #pragma once
00002 #ifndef OPTIONS_H
00003 #define OPTIONS_H
00004
00005 #include <stdio.h>
00006 #include <getopt.h>
00007 #include <string.h>
00008 #include <stdlib.h>
00009 #include <sys/stat.h>
00010 #include <limits.h>
00011 #include "../log/log.h"
00012 #include "context.h"
00013
00028 void print_info();
00029
00039 int parse_options(int argc, char **argv, struct cache_control *cc);
00040
00047 int validate_path_config(char *path);
00048
00051 #endif
```

7.10 Файл include/cache_control/parse_config.h

Заголовочный файл парсера конфигурации кэширования

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fnmatch.h>
#include <dirent.h>

#include <limits.h>
#include "../list/list.h"
Граф включаемых заголовочных файлов для parse_config.h: Граф файлов, в которые включается этот файл:

Макросы

- #define DEFAULT_INTERVAL 60
- #define SIZE_BUFF 4096

Функции

- int parse_config (const char *config_path, struct list *l, int *interval)
  Разбирает конфигурационный файл и заполняет список файлов

7.10.1 Подробное описание

Заголовочный файл парсера конфигурации кэширования

Версия

1.0

7.11 parse_config.h

См. документацию.

```
00001
00015 #ifndef PARSE_CONFIG_H
00016 #define PARSE_CONFIG_H
00017
00018 #include <stdio.h>
00019 #include <stdlib.h>
00020 #include <string.h>
00021 #include <fnmatch.h>
00022 #include <dirent.h>
00023 #include <limits.h>
00024 #include "../list/list.h"
00025
00026 #define DEFAULT_INTERVAL 60
00027 #define SIZE_BUFF 4096
00055 int parse_config(const char *config_path, struct list *l, int *interval);
00056
00057 #endif
00058
```

7.12 signal_handler.h

```
00001 #pragma once
00002
00003 #ifndef SIGNAL_HANDLER_H
00004 #define SIGNAL_HANDLER_H
00005
00006 #include <stdio.h>
00007 #include <signal.h>
00008 #include <stdlib.h>
00009 #include "context.h"
00010 #include "../log/log.h"
00011
00022 void sigint_handler(int sig);
00023
```

```
00029 void sigterm_handler(int sig);
00030
00036 void sighup_handler(int sig);
00037
00043 void sigusr1_handler(int sig);
00044
00052 int register_handler(int signum, void (*handler)(int));
00053
00061 void init_signal_handler();
00062
00069 void set_context(struct cache_control *cc);
00070
00073 #endif
```

7.13 threads.h

```
00001 #pragma once
00002
00003 #ifndef THREAD_H
00004 #define THREAD_H
00005
00006 #include "context.h"
00007 #include <pthread.h>
00008 #include "command_listener.h"
00009 #include "file_monitoring.h"
00010
00024 int start_threads(struct cache_control *cc);
00025
00031 int stop_threads();
00032
00035 #endif
```

7.14 Файл include/cache_control_mediator/cache_control_mediator.h

Заголовочный файл модуля управления кэшем через очередь сообщений

#include <unistd.h>
#include <string.h>
#include <stdio.h>
#include "../queue/queue.h"
#include <sys/stat.h>
#include "../list/list.h"
Граф включаемых заголовочных файлов для cache_control_mediator.h:

7.15 cache_control_mediator.h

См. документацию.

```
00001
00015 #ifndef CACHE_CONTROL_MEDIATOR_H
00016 #define CACHE_CONTROL_MEDIATOR_H
00017
00018 #include <unistd.h>
00019 #include <string.h>
00020 #include <stdio.h>
00021 #include "../queue/queue.h"
00022 #include <sys/stat.h>
00023 #include "../list/list.h"
00024
00025
00026 #define SEND_FILE "/snd_queue"
00027 #define RECV_FILE "/rcv_queue"
00028 #define CLEAR_TERMINAL "\033[2J\033[H"
00029 #define RED "\033[31m"
00030 #define DASHES "--------------------------\n"
00031 #define N 255
00032 #define MAX_TOK 2
00033 #define PATH_LEN 255
00034 #define COMMAND_LEN 10
```

```
00035 #define PATH_BUFF_SIZE 255
00036 #define PATH_LOG_FILE "~/logs/cache_control_mediator.log"
00037 #define POLITIC_LEN 15
00043 typedef struct {
00044     char command[COMMAND_LEN];
00045     char path_to_file[PATH_LEN];
00046 } command_t;
00047
00049 struct file_info {
00050     char path[PATH_BUFF_SIZE];
00051     off_t size;
00052     int number_of_pages;
00053     int number_of_pages_cache;
00054     int percent_cache;
00055     struct timespec last_access;
00056     int politic;                    /* Политика кэширования */
00057     int age;
00058 };
00059
00063 typedef struct {
00064     int status;          /* Статус выполнения: 0 - успешно, 1 - файл не найден */
00065     int last_msg;        /* Флаг последнего сообщения: 0 - последнее, 1 - есть ещё данные */
00066     struct file_info f; /* Статистика по файлу (заполняется при status) */
00067 } reply_t;
00068
00069
00074 enum pol {
00075     P_KEEP = 1,
00076     P_EVICT,
00077     P_MONITOR
00078 };
00079
00084 typedef struct {
00085     mqd_t fds[2];
00086 } mqfd_t;
00087
00092 typedef enum {
00093     CMD_EXIT,
00094     CMD_RELOAD,
00095     CMD_EVICT,
00096     CMD_CACHE,
00097     CMD_STATUS,
00098     CMD_CLEAR,
00099     CMD_UNKNOWN
00100 } command_type_t;
00101
00107 int read_string(char *str);
00108
00113 void clear_stdin();
00114
00119 void print_menu();
00120
00128 int get_command(char *str, command_t *command, command_type_t *op);
00129
00137 int print_answer(const char *const answer, const command_t *const ptr_command,
00138         const command_type_t op);
00139
00148 int split_tokens(char *str, char *tokens[], const char *delim,
00149         size_t *count_tok);
00150
00159 mqfd_t open_queue(const char *send_path, const char *recv_path,
00160          const int send_flags, const int recv_flags);
00161
00170 int set_command(const char *str, command_t *command, const char *path,
00171         const command_type_t op);
00172
00178 command_type_t get_command_type(const char *cmd);
00179
00186 int get_politic_str(reply_t *replay, char *str);
00187
00188 #endif
00189
```

7.16 list.h

```
00001 #pragma once
00002
00003 #ifndef LIST_H
00004 #define LIST_H
00005
00006 #include <malloc.h>
00007 #include <stdio.h>
00008 #include <string.h>
00009 #include <time.h>
```

```
00010 #include <unistd.h>
00011 #include <sys/mman.h>
00012 #include "../log/log.h"
00013
00019 #define PATH_BUFF_SIZE 255 /* размер буфера пути файла */
00020
00026 enum politics { KEEP = 1, EVICT, MONITOR };
00027
00031 struct file_stat {
00032     off_t size; /* рамер файла */
00033     int number_of_pages; /* общее количество страниц */
00034     int number_of_pages_cache; /* количество занятых страниц */
00035     int percent_cache; /* процентное соотщение занятых страниц к общему количеству */
00036     struct timespec last_access; /* время последнего доступа к файлу */
00037 };
00038
00042 struct file {
00043     char path[PATH_BUFF_SIZE]; /* путь файла */
00044     struct file_stat *stat; /* статистика файла */
00045     int politic; /* политика */
00046     int age; /* время через, которое файл будет выгружен при политике EVICT */
00047     int wd; /* дескриптор просмотра (watch descriptor) */
00048     int fd; /* файловый дескриптор */
00049     void *vmem_mapping; /* указтель на виртуальную память*/
00050 };
00051
00055 struct node {
00056     struct file *nd_file; /* указатель на файл */
00057     struct node *nd_next; /* указатель на следующую вершину в списке */
00058 };
00059
00063 struct list {
00064     struct node *start; /* указатель на первую вершину в списке*/
00065     int length; /* размер списка */
00066     int evict_files; /* количество evict файлов*/
00067 };
00068
00080 struct file_stat *init_file_stat(off_t s, int np, int npc, int pc,
00081                 struct timespec *la);
00082
00090 struct file_stat *init_empty_stat();
00091
00101 struct file *init_file(char *pb, int p, int a);
00102
00109 int add_file_stat(struct file *f, struct file_stat *fs);
00110
00117 struct node *init_node(struct file *f);
00118
00125 struct list *init_list();
00126
00133 int add_file(struct list *l, struct file *f);
00134
00141 struct file *find_file_by_path(struct list *l, char *path);
00142
00149 struct file *find_file_by_wd(struct list *l, int wd);
00150
00157 int delete_file_by_path(struct list *l, char *path);
00158
00166 int delete_file_by_wd(struct list *l, int wd);
00167
00173 int free_list(struct list *l);
00174
00177 #endif
```

7.17 log.h

```
00001 #ifndef LOG_H
00002 #define LOG_H
00003 #include <stdio.h>
00004 #include <string.h>
00005 #include <syslog.h>
00006 #include <errno.h>
00012 #define LOG_BUFF 512 /*размер буфера для лог сообщений */
00016 void open_log();
00017 /*
00018  * \brief Функция записи сообщения в syslog
00019  * \details функция записывает сообщение в syslog  с определенным уровнем логирования
00020  * \param int log_level - указывает уровани логирования для сообщения используются только LOG_INFO,
      LOG_WARNING, LOG_ERR
00021  * \param char *msg - сообщение которое будет записано в syslog
00022  * \warning при использовании LOG_ERR сообщение будет дополненно информацией из errno
00023  */
00024 void logging(int log_level, char *msg);
00028 void close_log();
```

```
00030 #endif
```

7.18 queue.h

```
00001 #ifndef QUEUE_H
00002 #define QUEUE_H
00003 #include <malloc.h>
00004 #include <stdio.h>
00005 #include <stdlib.h>
00006 #include <string.h>
00007 #include <unistd.h>
00008 #include <sys/stat.h>
00009 #include <fcntl.h>
00010 #include <mqueue.h>
00011 #include <time.h>
00012 #include <errno.h>
00018 #define SIZE_RCV 1024
00019 #define SIZE_SND 45
00020 #define MAX_SIZE_MSG 8192
00021 #define CHANGE_SIZE_MSG 8190
00022 #define MQ_TIMEOUT 5
00030 mqd_t create_queue(char *file_desc, int flags, int size_msg);
00038 char *recive_msg(mqd_t queue, char *snd_msg, int size_msg);
00045 int send_msg(mqd_t queue, char *snd_msg, int size_msg);
00053 char *recive_msg_for_mediator(mqd_t queue_fd, char *rcv_msg, int size_msg);
00055 #endif
```

Предметный указатель

add_all_files_to_inotify
    Inotify_stuff, 25
add_fd_to_epoll
    Event_stuff, 15
add_file
    List, 42
add_file_stat
    List, 42
age
    file_info, 53

cache_control, 51
CacheControl — система управления страничным кэшем, 1
CHANGE_SIZE_MSG
    Queue, 47
check_type_fd
    File_monitoring, 18
clear_stdin
    control mediator, 37
CLEAR_TERMINAL
    control mediator, 35
CMD_CACHE
    control mediator, 37
CMD_CLEAR
    control mediator, 37
CMD_EVICT
    control mediator, 37
CMD_RELOAD
    control mediator, 37
CMD_STATUS
    control mediator, 37
CMD_UNKNOWN
    control mediator, 37
com
    mediator_command, 56
command
    command_t, 52
COMMAND_LEN
    control mediator, 35
command_t, 51
    command, 52
    path_to_file, 52
command_type_t
    control mediator, 36
Contex, 13
    init_context, 13
    remove_context, 14
control mediator, 33
    clear_stdin, 37
    CLEAR_TERMINAL, 35
    CMD_CACHE, 37
    CMD_CLEAR, 37
    CMD_EVICT, 37
    CMD_RELOAD, 37
    CMD_STATUS, 37
    CMD_UNKNOWN, 37
    COMMAND_LEN, 35
    command_type_t, 36
    get_command, 37
    get_command_type, 38
    get_politic_str, 38
    MAX_TOK, 35
    N, 35
    open_queue, 38
    P_EVICT, 37
    P_KEEP, 37
    P_MONITOR, 37
    PATH_BUFF_SIZE, 35
    PATH_LEN, 35
    PATH_LOG_FILE, 36
    pol, 37
    POLITIC_LEN, 36
    print_answer, 39
    print_menu, 39
    read_string, 39
    RECV_FILE, 36
    RED, 36
    SEND_FILE, 36
    set_command, 40
    split_tokens, 40
create_queue
    Queue, 48

Daemonize, 14
    daemonize, 14
daemonize
    Daemonize, 14
DEFAULT_INTERVAL
    Parse_config, 29
delete_file_by_path
    List, 43
delete_file_by_wd
    List, 43
do_file_loading
    File_monitoring, 19
do_file_statistics
    File_monitoring, 19

Event_stuff, 15
    add_fd_to_epoll, 15
    set_timer, 15

f
    replay_t, 58
fds
    mqfd_t, 57
file, 52
file_in_attrib
    Inotify_stuff, 25
file_in_modify
    Inotify_stuff, 26
file_info, 53
    age, 53
    last_access, 53
    number_of_pages, 53
    number_of_pages_cache, 53
    path, 54
    percent_cache, 54
    politic, 54
    size, 54
File_loading, 16
    load_file, 16
    map_file, 17
    unmap_file, 17
File_monitoring, 17
    check_type_fd, 18
    do_file_loading, 19
    do_file_statistics, 19
    init_file_monitoring, 19
    map_all_files, 20
    processing_all_files, 20
    reload_config, 21
    start_file_monitoring, 21
file_monitoring, 54
file_stat, 55
File_statistics, 22
    get_file_size, 22
    get_last_access, 22
    get_number_of_pages, 23
    get_number_of_pages_cache, 23
    get_page_stat, 24
    get_percent_cache, 24
find_file_by_path
    List, 43
find_file_by_wd
    List, 44
free_list
    List, 44

get_command
    control mediator, 37
get_command_type
    control mediator, 38
get_file_size
    File_statistics, 22
get_last_access
    File_statistics, 22
get_number_of_pages
    File_statistics, 23
get_number_of_pages_cache
    File_statistics, 23
get_page_stat
    File_statistics, 24
get_percent_cache
    File_statistics, 24
get_politic_str
    control mediator, 38

include/cache_control/command_listener.h, 59
include/cache_control/context.h, 59
include/cache_control/daemonize.h, 60
include/cache_control/events_stuff.h, 60
include/cache_control/file_loading.h, 60
include/cache_control/file_monitoring.h, 61
include/cache_control/file_statistics.h, 61
include/cache_control/inotify_stuff.h, 62
include/cache_control/options.h, 62
include/cache_control/parse_config.h, 62, 63
include/cache_control/signal_handler.h, 63
include/cache_control/threads.h, 64
include/cache_control_mediator/cache_control_mediator.h, 64
include/list/list.h, 65
include/log/log.h, 66
include/queue/queue.h, 67
init_context
    Contex, 13
init_empty_stat
    List, 45
init_file
    List, 45
init_file_monitoring
    File_monitoring, 19
init_file_stat
    List, 45
init_list
    List, 46
init_node
    List, 46
init_signal_handler
    Signal_handler, 30
Inotify_stuff, 24
    add_all_files_to_inotify, 25
    file_in_attrib, 25
    file_in_modify, 26
    read_inotify, 26
    remove_all_files_from_inotify, 26

last_access
    file_info, 53
last_msg
    replay_t, 58
List, 41
    add_file, 42
    add_file_stat, 42
    delete_file_by_path, 43

delete_file_by_wd, 43
find_file_by_path, 43
find_file_by_wd, 44
free_list, 44
init_empty_stat, 45
init_file, 45
init_file_stat, 45
init_list, 46
init_node, 46
politics, 42
list, 55
load_file
File_loading, 16

map_all_files
File_monitoring, 20
map_file
File_loading, 17
MAX_SIZE_MSG
Queue, 47
MAX_TOK
control mediator, 35
mediator_command, 56
com, 56
path_to_file, 56
MQ_TIMEOUT
Queue, 47
mqfd_t, 56
fds, 57

N
control mediator, 35
node, 57
number_of_pages
file_info, 53
number_of_pages_cache
file_info, 53

open_queue
control mediator, 38
Options, 27
parse_options, 27
print_info, 28
validate_path_config, 28

P_EVICT
control mediator, 37
P_KEEP
control mediator, 37
P_MONITOR
control mediator, 37
Parse_config, 28
DEFAULT_INTERVAL, 29
parse_config, 29
SIZE_BUFF, 29
parse_config
Parse_config, 29
parse_options
Options, 27
path
file_info, 54
PATH_BUFF_SIZE
control mediator, 35
PATH_LEN
control mediator, 35
PATH_LOG_FILE
control mediator, 36
path_to_file
command_t, 52
mediator_command, 56
percent_cache
file_info, 54
pol
control mediator, 37
politic
file_info, 54
POLITIC_LEN
control mediator, 36
politics
List, 42
print_answer
control mediator, 39
print_info
Options, 28
print_menu
control mediator, 39
processing_all_files
File_monitoring, 20

Queue, 47
CHANGE_SIZE_MSG, 47
create_queue, 48
MAX_SIZE_MSG, 47
MQ_TIMEOUT, 47
recive_msg, 48
recive_msg_for_mediator, 49
send_msg, 49
SIZE_RCV, 48
SIZE_SND, 48

read_inotify
Inotify_stuff, 26
read_string
control mediator, 39
recive_msg
Queue, 48
recive_msg_for_mediator
Queue, 49
RECV_FILE
control mediator, 36
RED
control mediator, 36
register_handler
Signal_handler, 30
reload_config
File_monitoring, 21
remove_all_files_from_inotify
Inotify_stuff, 26

remove_context
Contex, 14
replay_t, 57
f, 58
last_msg, 58
status, 58
reply_t, 58

SEND_FILE
control mediator, 36
send_msg
Queue, 49
set_command
control mediator, 40
set_context
Signal_handler, 31
set_timer
Event_stuff, 15
sighup_handler
Signal_handler, 31
sigint_handler
Signal_handler, 31
Signal_handler, 30
init_signal_handler, 30
register_handler, 30
set_context, 31
sighup_handler, 31
sigint_handler, 31
sigterm_handler, 32
sigusr1_handler, 32
sigterm_handler
Signal_handler, 32
sigusr1_handler
Signal_handler, 32
size
file_info, 54
SIZE_BUFF
Parse_config, 29
SIZE_RCV
Queue, 48
SIZE_SND
Queue, 48
split_tokens
control mediator, 40
start_file_monitoring
File_monitoring, 21
start_threads
Threads, 33
status
replay_t, 58
stop_threads
Threads, 33

Threads, 32
start_threads, 33
stop_threads, 33

unmap_file
File_loading, 17
validate_path_config
Options, 28